

CIA-2
-Assignment

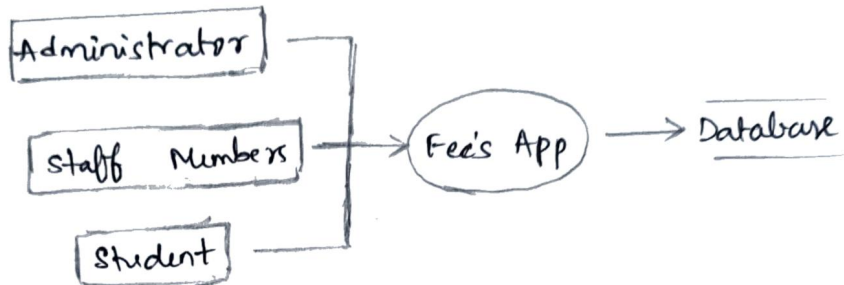
AP23110011349
Kavali Sindhu
CSE-F

→ Overview of the System :-

SRM's Fee Calculator (SFC) takes six inputs: Rank, Branch, Age, Accommodation type, Meal Option, and Gender. Based on predefined rules (tuition fee, accommodation fee, food charges, and discounts), it computes the total annual fee and displays it.

Level-wise DFD's :-

→ Level-0 (Context Diagram) :-



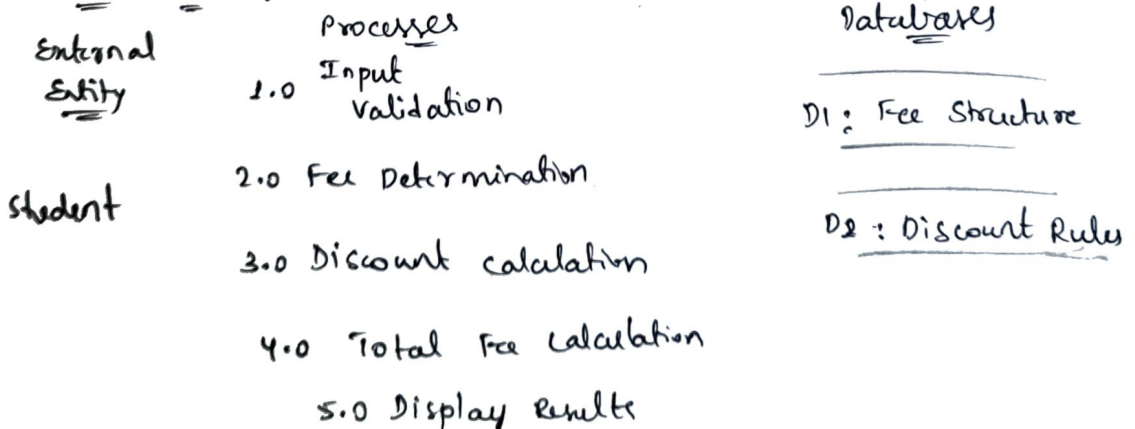
→ The entire system is represented as a single process: SFC System.

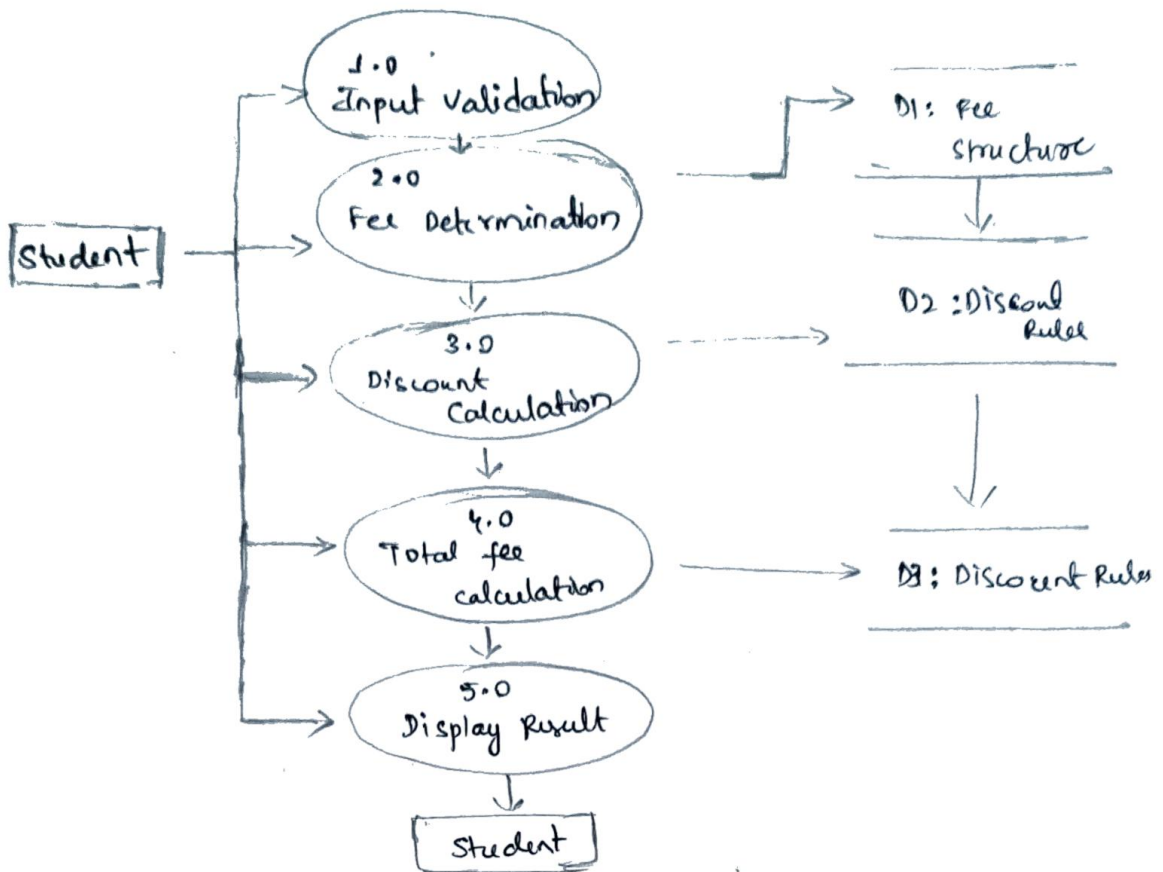
External Entities : Students / Admin / Staff

Input Data Flow : Rank, Branch, Age, Accommodation, Meal, Gender.

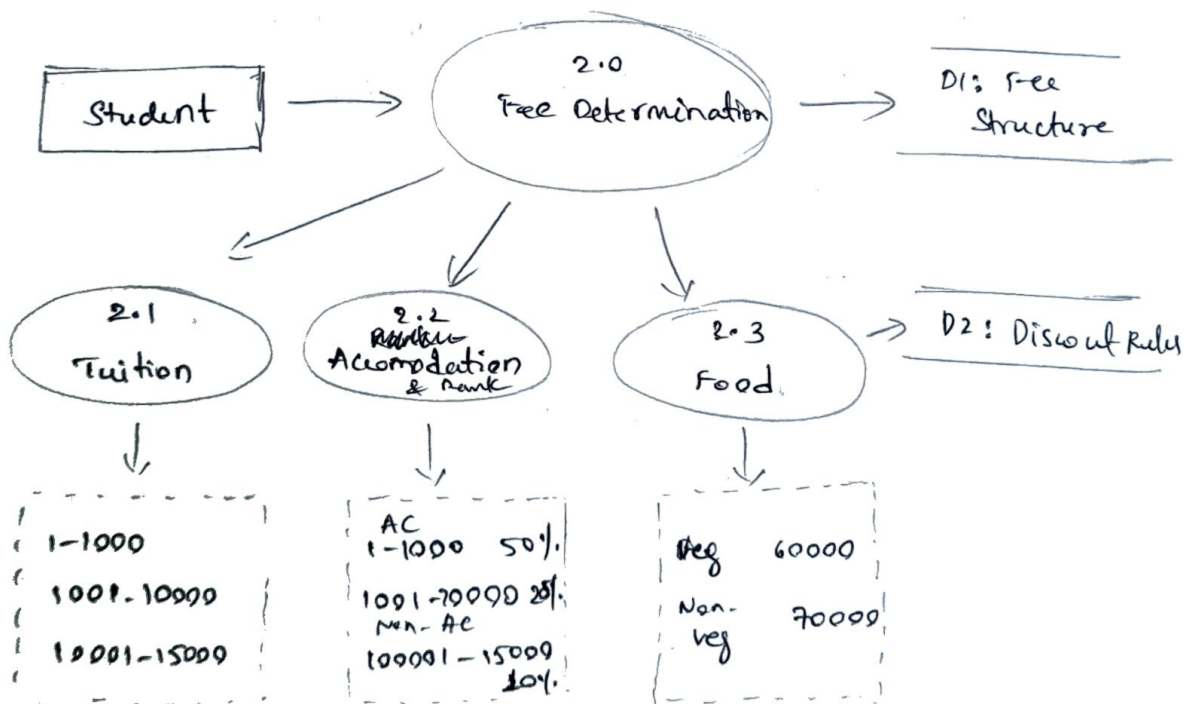
Output Data Flow : Total Annual fee

→ Level-1 DFD :-





Level-2 DFD :-



Data Dictionary :-

<u>Data Name</u>	<u>Description</u>
Rank	Student Entrance Rank
Branch	CSE/EEE/CIVIL/MECH
Age	Between 18-23
Accommodation	AC/NON-AC
Gender	Male/Female/Other
Tuition Fee	Based on branch
Discount	Based on rank & gender
Total Fee	Final fee

27

Cohesion & Coupling

Cohesion :- It refers to how closely related the functions within a module are.

Types :-

- 1) Functional (Best) : single task.
- 2) Sequential : output of one part used in next
- 3) Communicational : same data used.
- 4) Procedural : sequence-based tasks.
- 5) Temporal : time-based grouping
- 6) Logical : similar category tasks
- 7) Coincidental (worst) : unrelated tasks.

eg :- A module that only calculates tuition fee $\rightarrow$ High cohesion.

Coupling :- It refers to dependency b/w modules.

Types :-

- 1) Data Coupling (Best) : Only data passed.
- 2) Stamp Coupling : Passing structures.
- 3) Control Coupling : Passing control flags
- 4) External Coupling : Shared external systems.
- 5) Common Coupling : Shared global data.

Content coupling (worst): one module modifies another.

Eg:- If one module directly accesses discount module internal $\rightarrow$ high coupling (bad).

Low coupling & High cohesion is better, because,

1) High cohesion ensures modules are focused.

2) Low coupling ensures modules are independent.

Together they make software - flexible, scalable & reliable.

Advantages:-

1) Easy Maintainance

2) Better readability.

3) High reusability.

4) Easier debugging

5) Independent testing.

So, high cohesion & low coupling works best.

3) Approaches to Integration Testing:-

1) Top-Down Integration:-

i) Start from main module $\rightarrow$ go downward.

ii) Use stubs

Advantages:-

i) Early validation of high-level logic.

Disadvantages:

i) Lower modules tested late.

2) Bottom-up Integration:-

i) Start from lower modules $\rightarrow$ upward

ii) Use drivers.

Advantages :-

i) Core functionality tested early.

Disadvantages :-

i) No early system-level prototype.

3) Big Bang Integration :-

All modules are integrated at once.

Advantages :-

i) Simple approach.

Disadvantages :-

i) Hard to debug

ii) High risk.

4) Sandwich (Hybrid) Approach :-

Combination of top-down & bottom-up approach

Advantages :-

i) Balanced testing

ii) Faster error detection.

Integration testing ensures that modules interact correctly. The choice of approach depends on:

i) Project size.

ii) Risk level.

iii) Time constraints.